PYTHON PROGRAMMING

Module 3 – Program Control Flow

Conditional statements • Loops • Control statements • Assert

Learning objectives

By the end of this module, you will be able to:

Make decisions

Use if, if–else and if–elif–else.

Repeat actions

Use for and while loops.

Handle loop completion

Use for–else and while–else.

Control execution

Use break and continue.

Check assumptions

Use assert to detect failed assumptions.

Apply the concepts

Write and test programs combining conditions and loops.

Conditional statement: if

Runs an indented block only when its condition is True.

```
temperature = 32

if temperature > 30:
   print('It is hot')
   print('Stay hydrated’)

name = "Python"

if name == "Python":
   print("The string matches Python")
   print("Welcome to Python programming")
```

How it works

- Python evaluates the condition.
- If True, the indented block runs.
- If False, Python skips the block.
- Execution continues after the block.

Syntax: if condition: followed by an indented block

Conditional statement: if–else

Chooses one of two paths.

```
age = 20

if age >= 18:
   print('Adult')
   print('You can vote')
else:
   print('Not an adult’)
   print('You cannot vote')
```

Key points

- if block runs when the condition is True.
- else runs when the condition is False.
- else has no condition.
- Exactly one of the two blocks runs.

Output: Adult, You can vote

Conditional statement: if–elif–else

Conditions are checked in order; the first matching block runs.

```
#score = 76

score_input = input("Enter your score: ")
score = int(score_input)
if score >= 90:
    grade = 'A'
elif score >= 75:
    grade = 'B'
elif score >= 60:
    grade = 'C'
else:
    grade = 'D'

print(grade)
```

Evaluation

- score >= 90 → False
- score >= 75 → True
- Assign grade = 'B'
- Remaining branches are skipped.

Output: B

Choosing a conditional structure

Use the simplest structure that matches the decision.

if

One condition; action only when True.

```
if rain:
    take_umbrella
```

if–else

Two alternative paths.

```
if logged_in:
    show_page
else:
    show_login
```

if–elif–else

Several alternatives checked in order.

```
if score >= 90:
    grade = 'A'
elif score >= 75:
    grade = 'B'
else:
    grade = 'C'
```

The for Loop

What is a for loop?

A for loop repeats a block of code once for each item in a sequence, such as a list or string.

The loop variable takes each item in turn.

Basic syntax

```
for variable in sequence:
    statements
```

Example: Display a shopping list

```
shopping_list = ["Milk", "Bread", "Eggs", "Rice"]

for item in shopping_list:
    print(item)
```

Output: Milk Bread Eggs Rice (each on a new line)

How it works

1. The loop visits each item in shopping_list.
2. item holds the current item; print(item) displays it.

Remember: the statements inside the loop must be indented consistently.

Using range() in Python

range() generates a sequence of integers. The stop value is excluded.

1 range(stop)

```
for i in range(5):
    print(i)
```

OUTPUT

```
0, 1, 2, 3, 4
```

Starts at 0; stops before 5.

2 range(start, stop)

```
for i in range(2, 7):
    print(i)
```

OUTPUT

```
2, 3, 4, 5, 6
```

Starts at 2; stops before 7.

3 range(start, stop, step)

```
for i in range(2, 11, 2):
    print(i)
```

OUTPUT

```
2, 4, 6, 8, 10
```

Starts at 2; increases by 2; stops before 11.

4 Negative step

```
for i in range(10, 0, -2):
    print(i)
```

OUTPUT

```
10, 8, 6, 4, 2
```

Counts down by 2; stops before 0.

KEY POINTS stop is excluded • step controls the change • positive step counts up; negative step counts down

Looping: for–else

The else block runs when a for loop finishes normally, without break.

```
items = ['pen', 'book', 'ruler']
target = 'pencil'

for item in items:
    if item == target:
        print('Item found')
        break
else:
    print('Item not found')
```

Key points

- In this search, else reports that the target was not found.

Output: Item not found

The while Loop

Repeats a block of code as long as its condition is True.

Basic syntax

```
while condition:
    statements
```

How it works

1. Python checks the condition.
2. If True, the indented block runs.
3. The condition is checked again.
4. If False, the loop ends.

Example: Display numbers from 1 to 5

```
count = 1

while count <= 5:
    print(count)
    count += 1
```

OUTPUT

```
1    2    3    4    5
```

count += 1 updates the value so the loop can stop.

Remember: if the condition never becomes False, the loop may run indefinitely.

Common Uses of while

Choose the condition based on what should make the repetition stop.

1. Countdown

```
count = 5
while count > 0:
    print(count)
    count -= 1
```

Output: 5, 4, 3, 2, 1
Stops when count reaches 0.

2. Repeat until valid input

```
choice = ""
while choice != "yes":
    choice = input("Type yes: ")
print("Thank you")
```

Keeps asking until the user enters "yes".

3. Continue until a target

```
savings = 0
while savings < 500:
    savings += 100
    print(savings)
```

Output: 100, 200, 300, 400, 500
Stops when the target is reached.

4. Menu until Exit

```
choice = ""
while choice != "3":
    print("1. Add  2. View  3. Exit")
    choice = input("Choose: ")
```

Repeats the menu until the user enters 3.

Common pattern: initialise a value → test a condition → do the work → update the value or input.

while: Control Patterns and Common Mistakes

Use these patterns after you understand the basic condition-controlled loop.

while True with break

```
while True:
    answer = input("Exit? ")
    if answer == "yes":
        break
```

while True creates a loop that keeps running. break exits it when the chosen condition is met.

while–else

```
count = 1
while count <= 3:
    print(count)
    count += 1
else:
    print('Done')
```

The else block runs when the loop ends normally because its condition becomes False. It is skipped if break exits the loop.

Common mistakes to avoid

No update	The condition stays True; the loop may never stop.
Wrong condition	The loop may stop immediately or run too long.
Input not refreshed	A repeated input check must ask for input inside the loop.

Quick rule

for → iterate through items or a range
while → repeat while a condition is True

The break Statement

break immediately terminates the nearest enclosing loop when a condition is met.

1. break in a for loop

```
marks = [78, 65, 82, 28, 90, 75]

for val in marks:
    if val < 35:
        print("Failed mark found:",
mark)
        break

    print("Passed mark:", val)
```

```
OUTPUT
Passed mark: 78
Passed mark: 65
Passed mark: 82
Failed mark found: 28
```

HOW IT WORKS

Stops checking marks when a score below 35 is found. The remaining marks are not processed.

2. break in a while loop

```
count = 1

while count <= 10:
    print(count)

    if count == 5:
        break

    count += 1

print("Loop ended")
```

```
OUTPUT
1
2
3
4
5
Loop ended
```

HOW IT WORKS

Stops counting at 5, even though the while condition (count <= 10) is still True.

Remember: break exits only the nearest enclosing loop; execution continues after that loop.

The continue Statement

Skips the rest of the current iteration and moves to the next one.

Example: Skip marks below 35

```
marks = [78, 28, 65, 32, 82, 45]

for mark in marks:
    if mark < 35:
        continue
    print("Passed mark:", mark)

print("Checking completed")
```

Output

```
Passed mark: 78
Passed mark: 65
Passed mark: 82
Passed mark: 45
Checking completed
```

28 and 32 are skipped because they are below 35; the loop continues with the next mark.

Remember: continue = skip this iteration | break = exit the loop

Difference Between break and continue

Both control loop execution, but they behave differently.

Feature	break	continue
Purpose	Ends the loop immediately.	Skips the rest of the current iteration.
What happens next?	Execution moves to the first statement after the loop.	The loop moves to the next iteration.
Loop behavior	No further iterations are performed.	The loop continues with remaining iterations.
Typical use	Stop when the required item is found.	Skip items that should not be processed.
Example	Stop searching when a match is found.	Skip marks below 35 and display the rest.

Remember: break = stop the loop | continue = skip to the next iteration

The assert Statement

Checks whether a condition is true; if false, it raises an AssertionError.

Syntax

```
assert condition, "Error message"
```

Condition is True

Program continues normally.

Condition is False

Raises AssertionError.

Example: Validate Marks

```
marks = 85

assert 0 <= marks <= 100, "Marks must be between 0 and 100"

print("Marks are valid:", marks)
```

Output

```
Marks are valid: 85
```

If marks = 120:

```
AssertionError:
Marks must be between 0 and 100
```

Important points

- assert is useful for checking assumptions during development and debugging.
- The error message is optional.
- If an assertion fails, Python raises an AssertionError.
- Do not use assert for critical validation, such as validating user input or enforcing security rules, because assertions can be disabled when Python runs with optimization (-O).

Hands-on 1: conditional statements

Write, run and test each program.

Task A: Pass or fail	Print Pass if a score is at least 50; otherwise print Fail.
Task B: Grade	Use if–elif–else: A (90+), B (75–89), C (60–74), D (below 60).
Task C: Largest of three	Read three numbers and print the largest. Consider equal values.

Test normal values and boundary values such as 50, 75 and 90.

Hands-on 2: loops and control statements

Use for, while, break, continue and loop-else.

Task A: Total prices	Use a for loop to add item prices [120, 80, 150] and display the total.
Task B: Skip failed marks	Given marks [35, 72, 48, 91, 65], use continue to skip marks below 50.
Task C: Find an item	Search a shopping list; use break when the target is found and for–else to report Not found.
Task D: Running total	Use while to add numbers 1 through 5 to a running total; update the loop variable.

For each task: predict the output, run the program, then test another case.

Hands-on 3: Full Program – Guess the number game

Problem Statement:

1. Write a Python program to create a *Guess the Number* game.
2. The computer should randomly generate a secret number between **1 and 20**.
3. Display a welcome message asking the player to guess a number between 1 and 20.
4. Allow the player a maximum of **6 attempts** to guess the number.
5. For each attempt:
   - Ask the player to enter a guess.
   - If the guess is lower than the secret number, display **"Your guess is too low."**
   - If the guess is higher than the secret number, display **"Your guess is too high."**
   - If the guess is correct, display a success message and stop asking for further guesses.
6. If the player guesses correctly, display the number of attempts taken.
7. If the player fails to guess the number within 6 attempts, reveal the secret number and display an appropriate message.

Appendix: Hands-on 1 — Answers

Conditional statements

Task A: Pass or fail

```
score = int(input("Enter score: "))

if score >= 50:
    print("Pass")
else:
    print("Fail")
```

Example: 65 → Pass

Task B: Grade

```
score = int(input("Enter score: "))

if score >= 90:
    print("Grade A")
elif score >= 75:
    print("Grade B")
elif score >= 60:
    print("Grade C")
else:
    print("Grade D")
```

Example: 82 → Grade B

Task C: Largest of three

```
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))
c = int(input("Enter third number: "))

if a >= b and a >= c:
    print("Largest number:", a)
elif b >= a and b >= c:
    print("Largest number:", b)
else:
    print("Largest number:", c)
```

Example: 25, 40, 40 → Largest number: 40

Appendix: Hands-on 2 — Answer Key

Loops and control statements

Task A: Total prices

```
prices = [120, 80, 150]
total = 0

for price in prices:
    total += price

print("Total price:", total)
```

Output: 350

Task B: Skip failed marks

```
marks = [35, 72, 48, 91, 65]

for mark in marks:
    if mark < 50:
        continue
    print(mark)
```

Output: 72, 91, 65

Task C: Find an item

```
items = ["Milk", "Bread", "Eggs", "Rice"]
target = input("Search item: ")

for item in items:
    if item.lower() == target.lower():
        print("Item found:", item)
        break
else:
    print("Item not found")
```

Try: Eggs / Butter

Task D: Running total

```
number = 1
total = 0

while number <= 5:
    total += number
    number += 1

print("Running total:", total)
```

Output: 15

Hands-on 3: Answer – Guess the number game

```
# This is a guess the number game.

import random

secretNumber = random.randint(1, 20)

print("Secret number (for testing):", secretNumber) #Just to check the number for testing purpose

print('I am thinking of a number between 1 and 20.')

# Ask the player to guess 6 times.
for guessesTaken in range(1, 7):
    print('Take a guess.', guessesTaken)
    guess = int(input())

    if guess < secretNumber:
        print('Your guess is too low.')
    elif guess > secretNumber:
        print('Your guess is too high.')
    else:
        break  # This condition is the correct guess!

if guess == secretNumber:
    print('Good job! You guessed my number in ' + str(guessesTaken) + ' guesses!')
else:
    print('Nope. The number I was thinking of was ' + str(secretNumber))
```